

BACKGROUND & MOTIVATION

Reinforcement Learning (RL) is a machine-learning architecture that uses a reward system to help train a model to take actions in a training environment as seen in Figure 1. Deep Reinforcement Learning (DRL), while similar to RL, focuses on making increasingly better decisions over time. In this project, DRL was chosen to improve an autonomous vehicle model with the goal to eventually be able to promise safer and more efficient transportation solutions for the future.

PURPOSE AND HYPOTHESIS

Purpose: The purpose of this presentation is to showcase the development and training of a deep reinforcement learning (DRL) model to enable autonomous driving in the CARLA simulator. We aim to demonstrate the effectiveness and potential of DRL techniques in creating intelligent agents capable of navigating complex environments and adhering to traffic rules.

Hypothesis: We hypothesize that by utilizing DRL algorithms, we can train a model to effectively learn and adapt to various driving scenarios encountered in the CARLA simulator. We expect that the trained agent will demonstrate proficiency in tasks such as lane following, traffic light recognition, pedestrian avoidance, and safe navigation through intersections. Furthermore, we anticipate that the DRL-based autonomous driving system will exhibit robustness and adaptability to handle unforeseen challenges and dynamic environments.

TRAINING CHALLENGES

Challenges in Training:

- Many parameters to change.
- Lengthy training time.
- Optimizing reward function, steering, and throttle.
- Steering lock for early dropping.
- Changing simulation environment for better collision detection.
- Episode length, goal location, spawn location, etc.

Components of an AV:

- Throttle
- Speed
- Steering
- Braking (not utilized)
- Front facing RGB or SEM camera
- GPS location

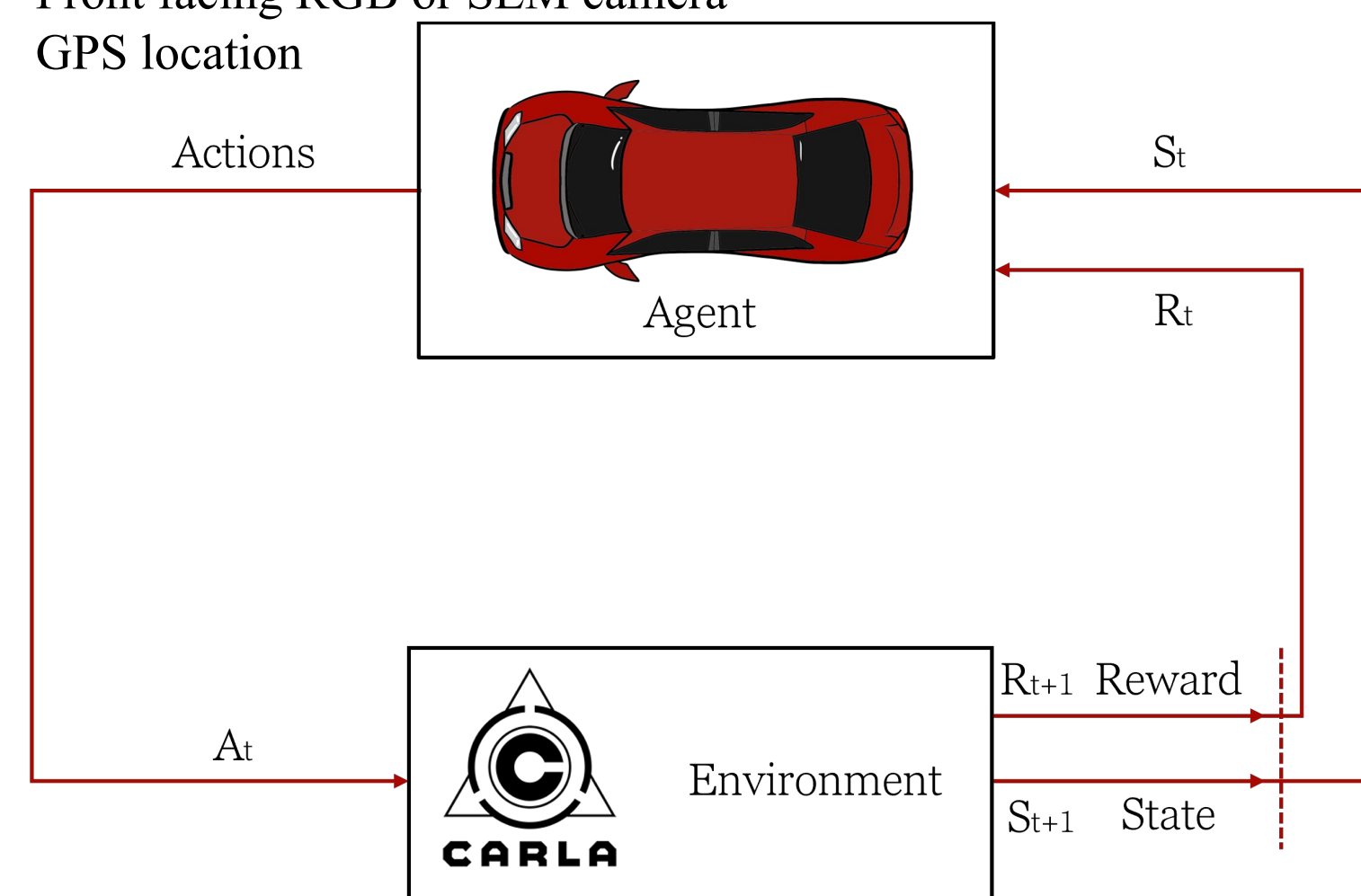


Figure 1: Reinforcement Learning Flow Chart.

METHODOLOGY

The Proximal Policy Optimization (PPO) algorithm helps the agent, a vehicle in this case, learn through policy gradient methods. It learns a policy that is updated gradually as the agent has new experiences. PPO ensures that the policy follows the “proximal” aspect, meaning that the updated policy is similar to the previous policy, with minimal changes, so as not to have a drastic shift in policy with each new iteration. Additionally, PPO's stability and consistency make it particularly well-suited for training autonomous vehicle models, where smooth and reliable decision-making is crucial for safe navigation in dynamic environments. This algorithm is widely used in research pertaining to autonomous vehicle training with reinforcement learning because of this. By maintaining a balance between exploration and exploitation while ensuring gradual policy updates, PPO enables autonomous vehicles to learn different ways to adapt effectively in various driving conditions.

ENVIRONMENT

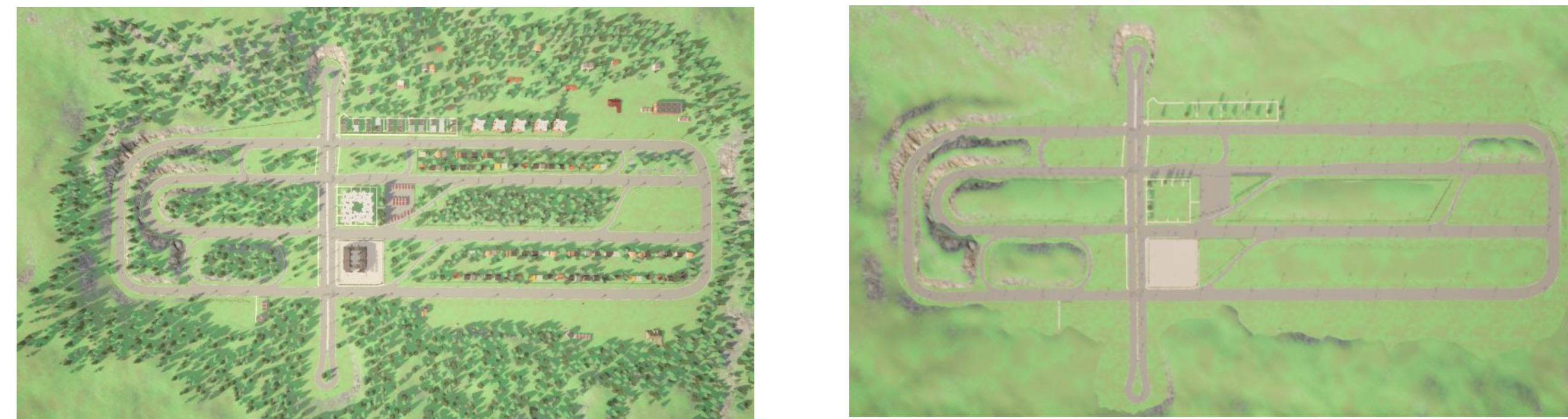


Figure 2: CARLA Town 6 unmodified (left) and modified to be a simpler environment (right).

The learning environment utilized for the agent was within the Car Learning to Act simulator, henceforth referred to as CARLA. CARLA offers a selection of different worlds with multiple terrains for training agents, along with a variety of Python functions to streamline environment setup. Initially, the environment spawned the vehicle at over 400 different points across the map, resulting in an overwhelming amount of information for effective learning. Consequently, this approach yielded suboptimal rewards as the model struggled to extract meaningful insights. The model interacted with the environment through input from an SEM camera mounted on top of the vehicle, providing segmented views of the path ahead. To enhance the model's performance, several adjustments were made. Initially, the focus was on refining the environment by reducing the number of spawn points while extending the episode length. This modification aimed to provide a more manageable learning experience for the model, which worked well. Additionally, we introduced different goals for each individual spawn point, four in total. If the vehicle achieved these goals, it would receive a large reward and then terminate the episode. Furthermore, to incentivize better navigation, more rewards for increased distance covered and proximity to the goal were incorporated. To mitigate unintended behaviors, such as veering off-course onto the big grassy areas, we manually added barriers to Town 6 to simulate crashes upon contact with the grass. This decision led to desired driving behavior. Initially, steer lock duration was very short which led to the premature termination of the episodes. Therefore, steer lock duration was increased, but at the cost of adding more negative reward if it lasted longer than the set amount. Finally, to promote safer driving of the agent, acceleration values were adjusted to prevent excessively high speeds, thus reducing the frequency of crashes. These adjustments were crucial in refining the model's learning process and led to enhancing its overall performance in the simulation as shown in Figure 3.

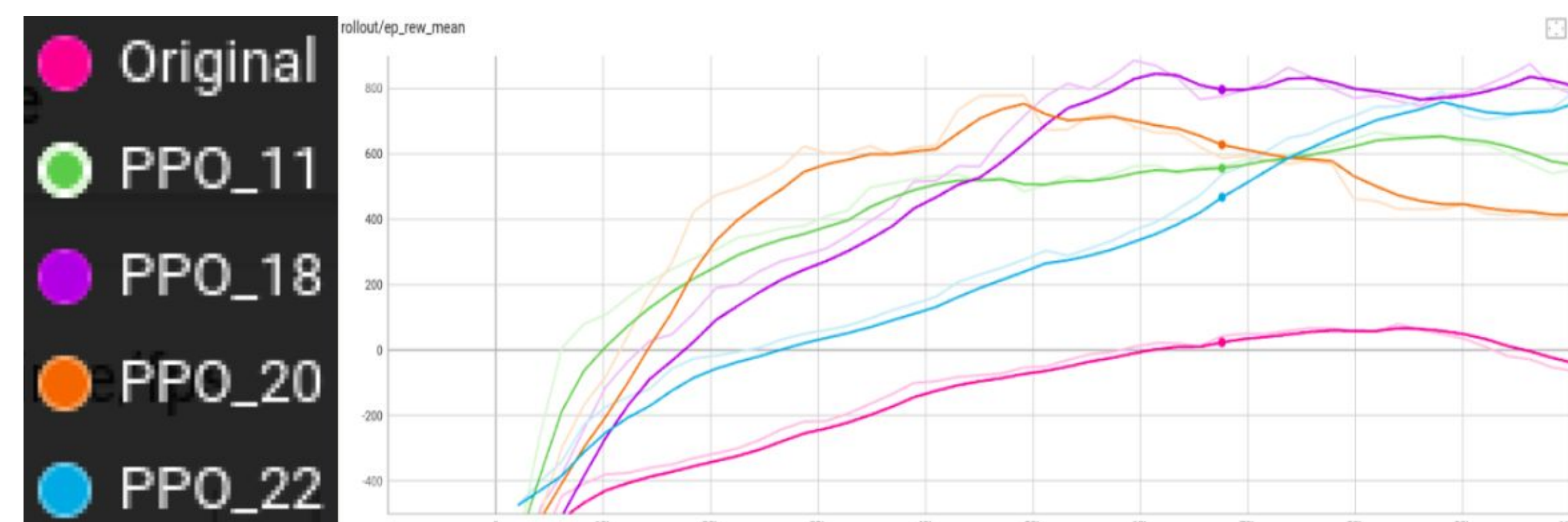


Figure 3: Reward curve comparisons, with PPO_18 being the highest of all.

CHANGES TO MODEL

- Longer episode length before stopping.
- Training on four specific spawn points instead of all available ones.
- Added goals for each spawn point.
- Modified map to be bare.
- Modified map to end episode faster by adding barriers.
- Added reward function to get reward for achieving goal and getting closer to it.
- Changed negative reward for collisions and steer lock to be more severe.
- Increased amount of time allowed for the car to steer lock before stopping episode.
- Allowed vehicle to be slightly further from goal and still count as achieving it.
- Changed and added accelerator values.

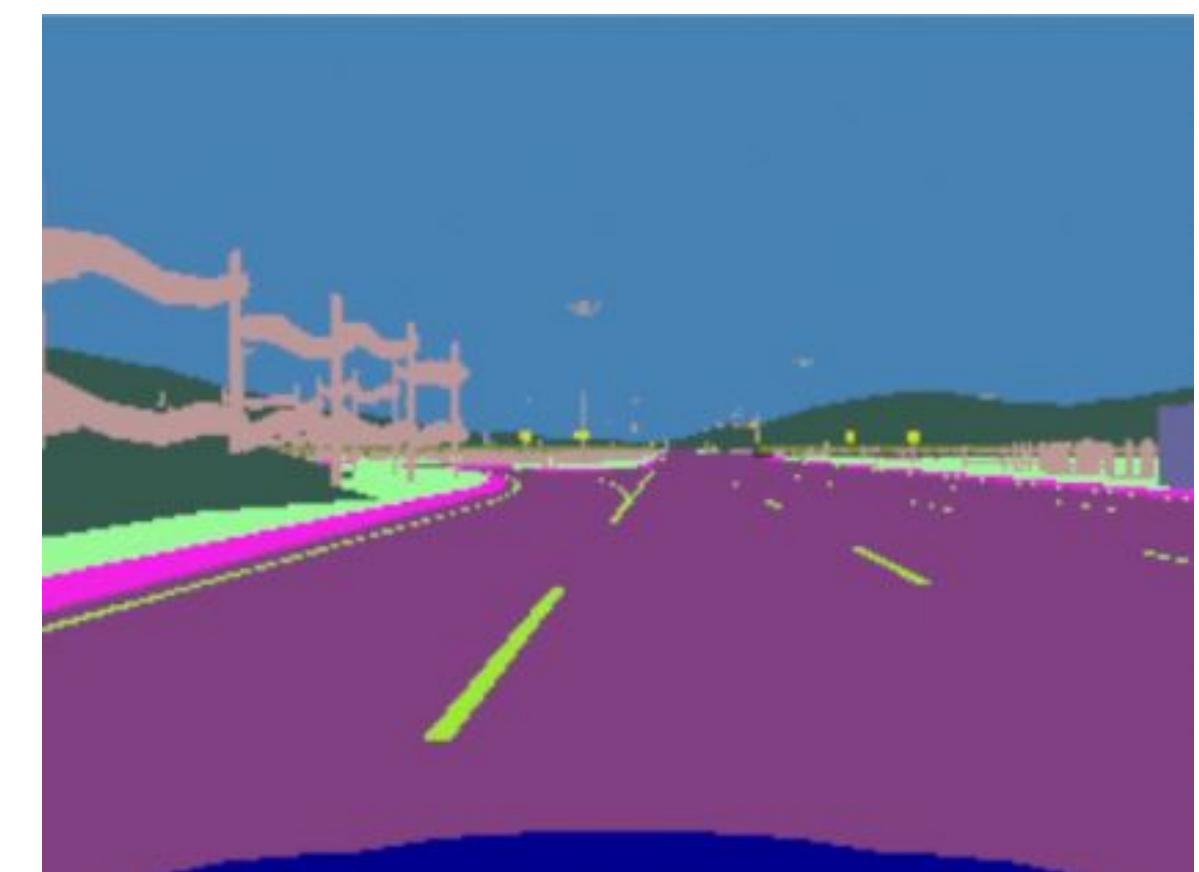


Figure 4: Still screenshot of PPO_18's SEM camera.

FUTURE WORK

1. Train to stay within road lines.
 - Train model further to get the agent to stay within road lines.
2. Train multiple models.
 - Train multiple models in the same environment.
 - Test multiple models in the same environment with incremental traffic and pedestrians.
3. Include more parameters: Breaking, maps, cars, pedestrians, etc.
 - Implement breaking to the model.
 - Train model in fully populated map environment.
4. Add checkpoints to environment.
 - Implement checkpoints and spawn locations as the model accuracy increases.

BIBLIOGRAPHY

CARLA, "Carla documentation," <https://carla.readthedocs.io/en/latest/>, February 2024, [Web page].

Hill, A., Raffin, A., Ernestus, M., Gleave, A., Kanervisto, A., Traore, R., ... Wu, Y. (2018). Stable Baselines. GitHub repository. Retrieved from <https://github.com/hill-a/stable-baselines>

Vadim7s. 2024. SelfDrive RL. GitHub. Retrieved from <https://github.com/vadim7s/SelfDrive/tree/master/RL>.